

FMCW Lab Feature Guide

Project: STM32 FMCW radar PC-side processing tools

Application folder: `tools/fmcw\_lab`

Firmware stream used: existing USB HS stream, VID/PID `1209:4158`, bulk IN endpoint
`0x81`

1. What Was Built

The `tools/fmcw\_lab` application is a PC-side radar workbench. It does not change the STM32 firmware. It connects to the radar over the same USB stream that was already tested with `fmcw\_stream\_read.py`, decodes each streamed ADC slope frame, and runs the radar DSP on the PC.

The implemented modules are:

File	Purpose
--- ---	
`usb_stream.py`	Opens the USB HS device, starts streaming, parses frame headers, and decodes 12-bit ADC samples.
`dsp.py`	Range FFT, range-Doppler, peak range detection, phase tracking, breathing/heartbeat analysis.
`sar.py`	Experimental SAR aperture FFT and monostatic backprojection helpers.
`app.py`	PyQtGraph GUI with connect/disconnect control and live visualizations.
`run.py`	Launch script for the app.

Run:

```
python .\tools\fmcw_lab\run.py
```

Dependencies:

```
python -m pip install pyusb libusb-package numpy pyqtgraph PyQt6
```

Only one PC program can own the USB interface at a time. Close other readers before opening this app.

2. Input Data Format

The current firmware sends one USB frame per captured DCMI/DMA slope frame.

Important constants from ``Core/Inc/fmcw_stream.h``:

Field	Value
USB frame size	<code>`20480`</code> bytes
Header size	<code>`64`</code> bytes
ADC payload	<code>`5000`</code> 32-bit DCMI words
ADC samples per frame	<code>`10000`</code> samples
ADC sample rate	<code>`10 MHz`</code>
ADC format	12-bit two's-complement

Each 32-bit DCMI word contains two ADC samples:

```
sample0 = word[11:0]
sample1 = word[27:16]
```

The PC app sign-extends each 12-bit value. Since the LTC1420 was treated as 4.096 V reference bipolar output, the current convention is:

```
1 ADC code ~= 1 mV
full-scale input range ~= +/-2.048 V
```

3. Range Measurement

How It Is Produced

For each selected chirp/slope frame:

1. Decode `10000` ADC samples.
2. Remove the per-chirp mean to suppress DC/leakage.
3. Apply a Hann window.
4. Run a real FFT.
5. Convert FFT beat frequency to range.

The range equation is:

$$\text{range_m} = c * \text{beat_frequency_hz} / (2 * \text{chirp_slope_hz_per_s})$$
$$\text{chirp_slope_hz_per_s} = \text{bandwidth_hz} / \text{chirp_time_s}$$

For the normal preset:

$$\text{bandwidth} = 200.271606 \text{ MHz}$$
$$\text{chirp time} = 1.000 \text{ ms}$$
$$\text{slope} = 200.271606 \text{ GHz/s}$$

Range resolution is approximately:

$$\text{delta_R} = c / (2 * \text{bandwidth})$$

For 200.271606 MHz, this is about `0.75 m`.

Suitable Chirp Settings

Goal	Recommended Chirp
General range bring-up	1 ms, 200 MHz

Better range resolution	Increase bandwidth
Lower noise floor per FFT bin	Longer chirp, if hardware timing supports it
Very close-range debugging	1 ms is fine; reduce displayed range window
Fast motion experiments	Shorter chirp can help, but must match firmware capture length

For range measurement, bandwidth mainly sets range resolution. Chirp time mainly sets beat-frequency scaling, FFT bin width, and how much time you integrate per range profile.

4. Range-Doppler Processing

How It Is Produced

The app processes one slope direction at a time, usually ``slope_id = 0``.

1. Run range FFT for many consecutive selected-slope chirps.
2. Keep the complex range FFT values, not just magnitudes.
3. Stack them into a CPI.
4. Optionally subtract the slow-time mean to suppress stationary clutter.
5. Apply a slow-time window.
6. Run FFT across chirps for each range bin.
7. Display Doppler bins versus range.

Velocity is estimated from Doppler frequency:

$$\text{velocity_m_s} = \text{doppler_hz} * \text{wavelength_m} / 2$$

At 5.8 GHz:

wavelength $\approx$ 51.7 mm

Suitable Chirp Settings

Goal	Recommended Chirp
Slow human/body motion	1 ms chirp, CPI 128-512 selected slopes
Faster targets	Shorter chirp or shorter triangle period
Better Doppler resolution	More chirps per CPI
Less latency	Fewer chirps per CPI

The tradeoff is simple:

Doppler resolution improves with longer CPI time.

Maximum unambiguous velocity improves with faster chirp repetition.

Because the firmware alternates slope directions, processing only one slope means the slow-time spacing is approximately one triangle period.

5. Range Waterfall

How It Is Produced

The waterfall is an integrated range profile history.

1. Collect recent complex range FFT frames.
2. Average power across a short CPI.
3. Convert to dB.
4. Append each integrated profile as a new row.

This is useful for seeing targets moving over range without needing to interpret Doppler velocity.

Suitable Chirp Settings

Use the same settings as ordinary range measurement. For human motion and indoor debugging, the normal 1 ms / 200 MHz chirp is a good starting point.

6. Phase-Based Micromotion

How It Is Produced

Micromotion uses the phase of one complex range FFT bin. It does not use raw ADC time-domain samples directly.

Pipeline:

ADC chirp -> range FFT -> choose target range bin -> complex phase
-> unwrap phase over slow time -> convert phase to displacement

The displacement equation is:

$$\text{displacement_m} = \text{unwrapped_phase_rad} * \text{wavelength_m} / (4 * \pi)$$

This is why the app needs a stable range bin. If the selected bin jumps between clutter and the body, the phase becomes meaningless.

Suitable Chirp Settings

Goal Recommended Chirp
--- ---
Breathing/micromotion 1 ms, 200 MHz
Better target isolation Higher bandwidth
Better phase SNR Stable chirp, coherent PLL, longer observation time
Fast vibration Faster chirp repetition helps

For phase sensing, chirp coherence and range-bin stability matter more than raw bandwidth. Bandwidth still helps isolate the target from nearby clutter.

7. Breathing Monitoring

How It Is Produced

Breathing is extracted from the micromotion displacement trace.

1. Track one target range bin.
2. Convert phase to displacement.
3. Remove slow drift.
4. Analyze the low-frequency band:

breathing band = 0.10 Hz to 0.70 Hz
= 6 to 42 breaths/min

The app estimates the strongest spectral peak in this band and reports breaths per minute.

Suitable Chirp Settings

Goal	Recommended Chirp
First breathing demo	1 ms, 200 MHz, one slope direction
Stable display	Process decimated slow-time stream, about 20-100 Hz
Close-range chest tracking	Set target range manually if auto-target locks to clutter

Breathing is slow. You do not need to process every chirp for the breathing rate. The app decimates the slow-time phase stream by default.

8. Heartbeat Monitoring

How It Is Produced

Heartbeat uses the same phase-derived displacement trace, but analyzes a higher frequency band:

heartbeat band = 0.80 Hz to 2.50 Hz
= 48 to 150 beats/min

Heartbeat is much weaker than breathing. It is easily hidden by body motion, breathing harmonics, poor target-bin selection, and RF/phase instability.

Suitable Chirp Settings

Goal	Recommended Chirp
--- ---	
First heartbeat attempt	1 ms, 200 MHz
Better heartbeat SNR	Stable stationary chest target, strong return, low clutter
Reduce latency	10-20 s analysis window
Improve confidence	Suppress breathing motion before heartbeat extraction

The current app implements a first useful heartbeat view. It is not yet a medical-grade heartbeat algorithm. The practical next step is breathing suppression/adaptive filtering after the breathing demo works reliably.

9. SAR Imaging Experiments

How It Is Produced

SAR needs many coherent range profiles while the radar moves along a known aperture.

The app implements two experimental views:

1. Aperture FFT quicklook.
2. Simple monostatic backprojection.

The capture process:

move radar along a straight path -> collect complex range FFTs
-> estimate platform step from speed and chirp spacing
-> backproject range profiles onto an x/range grid

This is useful for checking whether phase coherence and aperture motion are present. It is not a replacement for a controlled rail/encoder setup.

Suitable Chirp Settings

Goal	Recommended Chirp
First SAR experiment	1 ms, 200 MHz
Better cross-range image	Stable slow platform motion and known position
Better range resolution	Higher bandwidth
Better focus	Encoder/IMU position metadata or autofocus

SAR quality depends heavily on position knowledge. Approximate hand motion will produce a rough image or a smeared image. A rail or encoder is strongly preferred.

10. Firmware Timing: Is Variable Chirp Time Doable?

Short answer:

The ADF4158 can be programmed for variable chirp times,
but the current streaming firmware flow is effectively fixed around a
10000-sample / 1 ms capture frame at 10 MHz.

Why It Is Currently Fixed

The firmware stream constants are compile-time definitions:

```
#define FMCW_STREAM_SAMPLE_RATE_HZ    10000000UL
#define FMCW_STREAM_SAMPLES_PER_SLOPE 10000UL
#define FMCW_STREAM_WORDS_PER_SLOPE   5000UL
```

```
#define FMCW_STREAM_FRAME_BYTES      20480U
```

The DCMI DMA is also armed with:

```
DMA2_Stream1->NDTR = FMCW_STREAM_WORDS_PER_SLOPE;
```

That means every frame captures exactly `5000` DCMI words, or `10000` ADC samples. At 10 MHz, this is exactly:

```
10000 samples / 10 MHz = 1.000 ms
```

The USB parser, headers, buffer size, and PC app all assume this fixed frame shape.

What Happens If The PLL Chirp Time Does Not Match?

If the PLL chirp is shorter than 1 ms but firmware still captures 10000 samples:

```
one USB frame contains more than one chirp
range FFT becomes invalid for a single chirp
phase tracking and Doppler axes become wrong
```

If the PLL chirp is longer than 1 ms:

```
one USB frame only captures the first 1 ms of the chirp
bandwidth used by DSP is not the full programmed bandwidth
range scaling becomes wrong unless corrected
```

Important observation from the current code snapshot:

```
ADF4158_ProgramFastChirpTest();
```

appears in the streaming branch of `main.c`. That fast test preset is documented as a `0.08 ms` chirp. If the firmware is run exactly like that, the stream capture length no longer matches the chirp. For the PC app defaults, use the normal 1 ms preset or change the PC/firmware timing together.

Is Variable Chirp Time Possible Later?

Yes, but it needs a firmware-level refactor. The design should become configuration-driven:

1. Program the ADF4158 chirp preset.
2. Compute:

```
samples_per_slope = sample_rate_hz * chirp_time_s
words_per_slope = samples_per_slope / 2
payload_bytes = words_per_slope * 4
frame_bytes = header + payload + USB alignment padding
```

3. Reconfigure DMA `NDTR` for the selected chirp.
4. Update headers with the actual sample count and chirp parameters.
5. Make the PC parser accept variable frame sizes or a negotiated mode.
6. Ensure USB throughput can handle the selected frame rate.
7. Recheck MUXOUT alignment and HSYNC/VSNC timing for every mode.

So, variable chirp time is technically doable, but it is not currently an on-demand GUI setting. Right now it should be treated as fixed by the firmware build/preset.

11. Chirp Recommendations By Feature

Feature	Good Starting Chirp	Notes
Range measurement	1 ms, 200 MHz	Current normal preset; about 0.75 m resolution.
Range waterfall	1 ms, 200 MHz	Good for live bring-up and target movement.
Range-Doppler	1 ms, 200 MHz, CPI 128-512	More CPI improves velocity resolution but adds latency.
Micromotion	1 ms, 200 MHz	Stable phase and target-bin selection are most important.
Breathing	1 ms, 200 MHz	Decimate phase stream to 20-100 Hz.
Heartbeat	1 ms, 200 MHz	Needs strong stable chest return; heartbeat is weak.
SAR	1 ms, 200 MHz	Needs known aperture motion; bandwidth controls range resolution.
Fast motion tests	shorter chirp possible	Firmware capture length must be changed to match.
High range resolution	higher bandwidth	Requires corresponding ADF4158 values and front-end bandwidth.
Long-range low-noise tests	longer chirp can help	Firmware buffer length and USB rate must be updated.

12. Practical Bring-Up Order

Recommended order:

1. Use the normal 1 ms PLL preset.
2. Verify USB stream has no or low dropped-frame count.
3. Open the Range tab and confirm a stable peak.
4. Force the target range manually if auto-pick chooses leakage/clutter.
5. Check micromotion displacement.
6. Confirm breathing peak before attempting heartbeat.
7. Use range-Doppler once range FFT and stream health are good.
8. Treat SAR as a separate controlled experiment with known motion.

13. Reference Basis

This work was guided by:

- The local STM32 firmware stream implementation in ``Core/Inc/fmcw_stream.h`` and ``Core/Src/fmcw_stream.c``.
- The local ADF4158 presets in ``Core/Src/adf4158.c``.
- The ``ckflight/FMCW3`` project description, which lists the target feature set: range, range-Doppler, SAR, micromotion, breathing/heartbeat, USB streaming.

- The companion `ckflight/FMCW\_RADAR\_2v2` Python scripts, which use the same conceptual processing flow: range FFT, CPI range-Doppler, phase-bin tracking, and SAR experiments.